

Embeddings and Vector Databases

Technical reference for local semantic search

1. Text Embeddings

A text embedding is a dense numerical representation of text. The embedding model transforms a sentence, paragraph, or document into a fixed-length vector. Semantic similarity can then be estimated from vector distance or cosine similarity.

The embedding model used in this project is all-MiniLM-L6-v2. It is designed for sentence-level semantic representations and is small enough to run locally on ordinary development machines.

2. Cosine Similarity

Cosine similarity compares the angle between two vectors. A value close to 1 indicates strong directional similarity, a value near 0 indicates weak similarity, and negative values indicate opposing directions in spaces where negative similarity is possible.

$$\text{cosine_similarity}(a, b) = (a \cdot b) / (||a|| \ ||b||)$$

Vector databases often provide similarity-search APIs so application code does not need to implement the nearest-neighbor search itself.

3. ChromaDB Concepts

ChromaDB is a vector store that can persist embeddings and associated documents locally. A collection groups related records. Each record can include an identifier, document text, embedding, and metadata.

Field	Example	Role
id	chunk_001	Unique record identifier
document	text passage	Retrieved evidence
metadata	source/page	Traceability
embedding	numeric vector	Semantic search

4. Persistence

A persistent vector store keeps indexed data on disk so the application does not need to recompute embeddings every time it starts. A typical local project can use a directory such as `chroma_db` for persisted data.

```
vector_store = Chroma(  
    collection_name="technical_docs",  
    persist_directory="chroma_db",  
    embedding_function=embeddings,  
)
```

5. Indexing Strategy

The recommended indexing sequence is: load documents, split documents into chunks, create embeddings for each chunk, and add the chunks plus metadata to the vector store. The same embedding model must be used for both indexing and querying.

If documents are re-indexed repeatedly without stable identifiers, duplicate chunks may be created. Production systems should use deterministic IDs or collection replacement strategies.

6. Practical Trade-offs

Local vector databases provide simple development and privacy advantages. Their limitations can include memory usage, scaling constraints, and fewer distributed retrieval features compared with managed vector platforms.

For a student-scale technical Q&A; Copilot, ChromaDB is a practical starting point because it is easy to run locally and integrates with LangChain.